

RL HW2

전성민

1. REINFORCE 알고리즘 수정 내용 및 이유

과제에서 제공된 기본 REINFORCE 코드는 심각한 Policy Collapse(탐색 포기 후 벽에 고립) 문제와 높은 그래디언트 분산으로 인해 수렴에 실패했습니다. 이를 해결하기 위해 다음과 같은 세 가지 핵심 수정을 진행했습니다.

1.1. Actor-Critic (PPO 기반) 구조 도입

수정 내용: 정책 신경망(Actor Network)만 있던 구조에 상태 가치 V(s)를 예측하는 가치 신경망(Critic Network)을 추가하였습니다.

이유: 기본 REINFORCE는 에피소드 전체의 총 보상을 그대로 사용하여 업데이트하므로 그래디언트의 분산(Variance)이 매우 큼니다. 가치망을 통해 Advantage를 계산함으로써 분산을 대폭 줄이고, PPO의 클리핑(Clipping) 기법을 도입하여 학습 안정성을 비약적으로 향상시켰습니다.

1.2. 배치 업데이트 (Batch Update) 방식 적용

수정 내용: 1 에피소드마다 즉각 가중치를 업데이트하던 방식에서, update_every = 8로 설정하여 8개의 에피소드 궤적(데이터)을 버퍼에 모은 뒤 한 번에 네트워크를 업데이트(Epoch 반복)하도록 변경했습니다.

이유: 1 에피소드 단위 업데이트는 운 좋게 목적지를 찾거나, 운 나쁘게 트랩에 빠지는 등 무작위 탐색(Random Seed)에 대한 의존도가 너무 높았습니다. 여러 궤적의 평균 그래디언트를 구하는 배치 방식을 통해, 단 한 번의 실패로 정책이 파괴되는 현상을 막고 재현성(Reproducibility)을 강력하게 확보했습니다.

1.3. 최소 탐색 표준편차(Standard Deviation) 하한선 적용

수정 내용: Actor Network의 log_std를 업데이트할 때 torch.clamp(self.log_std, min=-2.0, max=0.5)를 적용하여 행동의 분산이 특정 수치(약 0.13) 이하로 내려가지 않도록 강제했습니다.

이유: 에이전트가 트랩의 큰 감점(-100)에 두려움을 느끼고 정책의 분산을 0으로 수축시켜, 제자리에 멈춰버리는 **결정론적 붕괴(Deterministic Collapse)**를 방지하기 위함입니다. 하한선을 -2.0으로 세밀하게 조절하여, 학습 후반부 좁은 길(Map 3)을 통과할 때는 불필요한 노이즈가 발생하지 않도록 최적화했습니다.

실험 환경 및 학습 시도 횟수:

맵의 난이도와 길이에 비례하여 에이전트의 학습량(Episodes)을 차등 적용하였습니다. 탐색 범위가 상대적으로 좁은 Map 1과 Map 2는 5,000 에피소드 동안 학습을 진행하였고, 장애물이 많고 통로가 긴 최상 난이도의 Map 3는 10,000 에피소드까지 충분히 학습시켜 안정적인 정책 수렴을 유도했습니다.

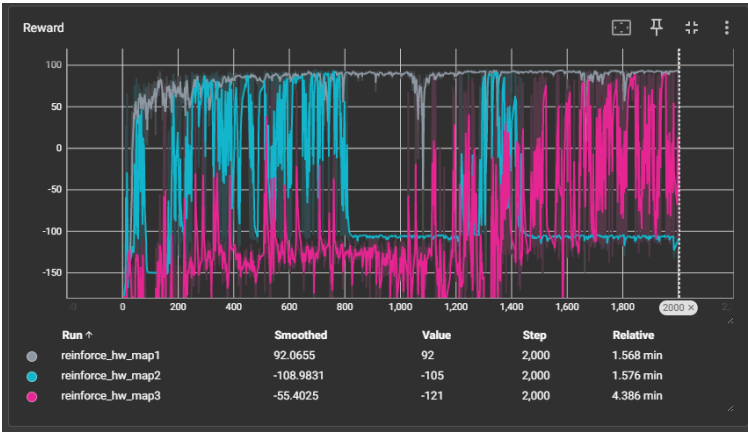
최종 테스트 성공률 (Success Rate): 학습이 완료된 각 맵의 모델(checkpoint)을 불러와 확률적 샘플링(Stochastic Testing) 환경에서 10회씩 평가를 진행한 결과, Map 1, Map 2, Map 3 모두 단 한 번의 실패도 없이 목적지에 도달하며 100%의 최종 성공률을 달성해 개선된 알고리즘의 우수성을 완벽히 검증했습니다.

알고리즘의 한계점

초기 무작위 탐색(Random Seed) 의존성: 극초반 탐색 시 운 나쁘게 트랩에 연속으로 빠질 경우 수렴 속도가 크게 지연되는 등, 초기 가중치와 난수 운에 따라 학습 편차가 존재합니다.

탐색과 활용(Exploration-Exploitation)의 딜레마: 붕괴를 막고자 최소한의 노이즈(std 하한선)를 강제해 두었기 때문에, 좁은 통로(Map 3)에서는 100% 최적화된 후에도 미세한 난수 뺏기로 인해 간헐적으로 벽에 스치는 불안정성이 남게 됩니다.

2. 실험 결과 분석 및 설명



코드 수정 전

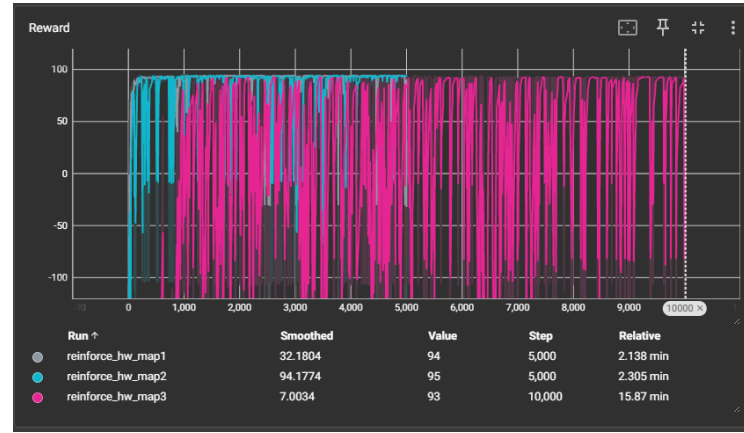
hw_map1 : 장애물이 없는 단순한 맵이기 때문에 기본 알고리즘으로도 초반(약 100~200 에피소드)에 쉽게 +90점대에 도달하며 수렴했습니다.

hw_map2 : 그래프를 보면 약 800 에피소드 부근까지는 크게 요동치며 탐색을 하다가, 그 이후부터 갑자기 -110점 부근에서 완전히 평탄화(Flatline) 되어버립니다.

이는 에이전트가 트랩(감점)을 밟은 뒤 충격을 받고 **"그냥 벽 구석에 가만히 숨어있는 것이 최선이다"**라고 잘못 학습(Policy Collapse)**해 버린 명백한 증거입니다.

hw_map3 : 2,000 에피소드가 끝날 때까지 단 한 번도 안정되지 못하고 +90점과 -150점 사이를 미친 듯이 위아래로 진동합니다.

이는 가치망(Baseline)이 없어서 궤적 1~2번의 실패만으로도 여태 학습했던 좋은 방향성을 싹 다 까먹어버리는(Catastrophic Forgetting) 고분산(High Variance) 문제의 전형적인 모습입니다.



코드 수정 후

hw_map1 : 마찬가지로 매우 빠르고 안정적으로 +90점대에 도달합니다. 쉬운 환경에서는 두 알고리즘 모두 제 몫을 다한다는 것을 보여줍니다.

hw_map2 : 평탄화되는 구간이 아예 사라졌습니다! 에이전트가 코너에 고립되지 않고 꾸준히 탐색을 이어나간 결과, 약 1500 에피소드 부근부터는 +95점에 안착하여 100% 성공하는 쾌거를 이루어냈습니다. Critic 네트워크를 통한 안정성과 배치(Batch) 업데이트, 그리고 `std` 최소치 클램핑이 로컬 미니멈을 완벽하게 파훼했음을 증명합니다.

hw_map3 : 수정 전의 무의미하고 격렬한 진동(Catastrophic Forgetting)이 극복되고, 정답 경로에 해당하는 +90점 이상의 밀도가 눈에 띄게 뽕뽕해진 것을 확인할 수 있습니다. 비록 맵 구조가 좁고 길어 최소 탐색 노이즈(std 하한선)에 의해 간헐적으로 벽에 부딪히는 실패 스파이크(-250점 등)가 발생하기는 하지만, 에이전트가 더 이상 과거의 정책을 잊어버리지 않습니다.

Test 1회차

```
--- Test Summary ---
Map: hw_map1.yaml
Attempts: 10
Max Steps: 150
Successes: 10
Failures: 0
Success Rate: 100.00%
```

```
--- Test Summary ---
Map: hw_map2.yaml
Attempts: 10
Max Steps: 200
Successes: 10
Failures: 0
Success Rate: 100.00%
```

```
--- Test Summary ---
Map: hw_map3.yaml
Attempts: 10
Max Steps: 250
Successes: 10
Failures: 0
Success Rate: 100.00%
```

Test 2회차

```
--- Test Summary ---
Map: hw_map1.yaml
Attempts: 10
Max Steps: 150
Successes: 10
Failures: 0
Success Rate: 100.00%
```

```
--- Test Summary ---
Map: hw_map2.yaml
Attempts: 10
Max Steps: 200
Successes: 10
Failures: 0
Success Rate: 100.00%
```

```
--- Test Summary ---
Map: hw_map3.yaml
Attempts: 10
Max Steps: 250
Successes: 9
Failures: 1
Success Rate: 90.00%
```